Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 - Machine Learning and Artificial Intelligence - Midterm Project, Fall 2026
Team leader: Daksh Mainee **Team member:** Sanan Goel

Summary. Can machine learning tell which large language model wrote a text? From the public M4 benchmark we built a balanced, topic-controlled dataset of 15,000 (LLM, prompt, response) triples: five LLMs (ChatGPT, GPT-3 davinci, Cohere, Dolly-v2, BLOOMz; four families) across five domains. We trained a CNN and a bidirectional LSTM. **RQ1:** from the response alone the CNN identifies the author 80.1% of the time and the BiLSTM 75.6% (chance 20%); a TF-IDF baseline reaches 88.9%. Most errors are between the two OpenAI models. **RQ2:** the prompt alone also identifies the model (72.0%), but we show this is fully explained by the per-model prompt templates used to build the dataset. **RQ3:** on a domain never seen in training, CNN accuracy drops by 23 points on average, partly because of a layout shortcut we identify. **RQ4:** the fingerprint is stylistic (function words, punctuation, layout, length) and highly redundant. With every content word masked, the CNN still reaches 0.81 macro-F1.

1. Problem Definition and Dataset Curation

1.1 Task

Each example is a triple *(LLM_name, LLM_input, LLM_output)*: the name of the model that wrote the text, the prompt it was given, and the response it produced. The task is **5-way single-label classification**: predict the LLM_name. RQ1 uses only the response; RQ2 compares a classifier that sees only the prompt, only the response, or both. Classes are perfectly balanced, so chance accuracy is 20%. We report **accuracy** and **macro-F1** (the unweighted mean of per-class F1, which drops sharply if a classifier ignores one model) on a held-out test set, plus confusion matrices and per-domain accuracy.

1.2 Data source

We used **M4** (Wang et al., EACL 2024, Best Resource Paper award), a public research benchmark of machine-generated text hosted at github.com/mbzuai-nlp/M4. We chose it over generating our own data because (i) it gives the same prompts to several LLMs, so topic is controlled, (ii) it spans five English domains, which makes RQ3 (cross-domain transfer) possible, and (iii) every record stores the exact prompt used, which RQ2 needs. We used the five English domains and the five generators that cover all of them:

- **ChatGPT** (gpt-3.5-turbo, OpenAI) and **GPT-3 davinci** (text-davinci-003, OpenAI) - two models from the *same* family, kept on purpose to test whether same-family models are harder to tell apart;
- **Cohere** (Cohere's generation API);
- **Dolly-v2** (dolly-v2-12b, Databricks; an instruction-tuned Pythia model from EleutherAI);
- **BLOOMz** (BigScience; multitask instruction-tuned BLOOM).

That is five models from four LLM families. Domains and what each model was asked to do:

Domain	Topic source	Task given to the LLM	ChatGPT	davinci	Cohere	Dolly	BLOOMz
wikipedia	Wikipedia titles	Write a Wikipedia article for the title	283	351	282	294	73
wikihow	WikiHow titles + headlines	Write a WikiHow how-to article	642	514	290	262	181
reddit	Reddit ELI5 / AskScience / AskHistorians questions	Answer the question in detail	254	312	140	247	159
arxiv	arXiv paper titles	Write the paper's abstract	215	131	201	211	167
peerread	PeerRead paper titles (+ abstracts)	Write a peer review (4 prompt variants)	290	172	144	351	135

Table 1. The five domains. Right-hand columns: median response length in words for each model.

1.3 Cleaning: problems we found in the raw data

Inspecting the raw files before training turned up several issues that would have produced misleading results:

- **BLOOMz echoes its prompt.** In the BLOOMz files the *machine_text* field is the prompt followed by the completion. Used as-is, every BLOOMz response would start with the instruction text and the classifier would get it right for free. We took the completion-only field (*machine_abstract* / *machine_answer*) and recovered the filled prompt by stripping the completion off the end of *machine_text*.
- **Empty generations.** 1,780 of the 3,000 Cohere Reddit responses are empty strings. We dropped every response with fewer than 10 words (this removed almost only those).
- **API whitespace.** Cohere responses begin with blank lines. That reflects how the API client returned the text, not how the model writes, so we stripped leading and trailing whitespace from every response.
- **Different storage formats.** PeerRead stores four (prompt, review) pairs per paper as lists, and BLOOMz stores its PeerRead prompts as unfilled templates. We expanded the lists into one example per pair and filled the templates with the paper's title and abstract.
- **Kept on purpose (and analysed in RQ3/RQ4):** Dolly-v2's responses contain no line breaks at all (0% of examples, versus 48-78% for the other models), which looks like the text was flattened during collection; and some BLOOMz outputs degenerate into repetition (e.g. a list of bare numbers "1. 2. 3. ..."), which is genuine model behaviour.

1.4 Sampling and train/validation/test split

We sampled **600 examples per (model, domain)**, giving **15,000** examples: 3,000 per model and 3,000 per domain, perfectly balanced. Two design decisions guard against shortcuts. **Topic-controlled sampling:** wherever possible we chose topics (the same Wikipedia title, question or paper) that were answered by all five models, so the subject matter carries no information about the label. This holds for 100% of the Wikipedia, WikiHow, Reddit and arXiv examples and 46.2% of PeerRead (fewer papers could be matched across files), 89.2% overall. **Grouped split:** we split by topic (70/15/15), so all five models' answers to a topic land in the same split and no test topic is ever seen in training. Result: 10,522 train, 2,244 validation and 2,234 test examples over 3,187 topics.

Data rights. M4 is a public research dataset released by its authors for machine-generated-text research. It contains LLM outputs and public prompts only (titles, public questions, public abstracts); no private, personal or confidential information. Our repository does not re-host the raw data: *src/build_dataset.py* rebuilds our exact sample from a clone of the M4 repository (fixed seed 42).

2. Model Implementation and Training

2.1 Input representation

We wrote a small regex tokenizer that keeps information many NLP pipelines throw away, because style lives there: **case is preserved**, every **punctuation and markdown symbol** is its own token, digits are kept, and every line break becomes a special <*nl*> token so layout is visible to the model. The vocabulary is built on the training split only (tokens seen at least twice, at most 30,000). Inputs: **output-only** = first 256 response tokens; **input-only** = first 128 prompt tokens; **input+output** = prompt (128) + <*sep*> + response (256). The median response is about 280 tokens, so 256 covers most of a typical response. Word embeddings are learned from scratch: pretrained embeddings were unavailable offline, and they are trained to ignore exactly the surface style we want to detect.

2.2 CNN

A Kim (2014)-style text CNN: embedding (128-d) → three parallel 1-D convolutions with kernel widths 3, 4 and 5 (100 filters each, ReLU), which act as learned n-gram detectors → global max-pooling over time (padding positions masked out) → concatenation (300-d) → dropout 0.5 → linear layer to 5 logits. Max-pooling makes the CNN ask *"does this phrase or pattern occur anywhere?"*, which suits fingerprint-like cues (3,995,789 parameters, 30,003-token vocabulary).

2.3 RNN (bidirectional LSTM)

Embedding (100-d, dropout 0.2) → one bidirectional LSTM layer with 64 units per direction, run on packed sequences so padding is never processed → the hidden states are summarised by concatenating mean-pooling and max-pooling over time (256-d) → dropout 0.5 → linear layer to 5 logits. Unlike the CNN, the LSTM reads the text in order and can model long-range structure such as how a response opens and closes (3,086,577 parameters).

2.4 Training procedure

Setting	Value
Loss / optimiser	Cross-entropy; Adam, learning rate 1e-3, no weight decay
Batch size / gradient clipping	64 (length-bucketed batches to reduce padding) / global norm 5.0
Epochs / early stopping	up to 10; stop when validation macro-F1 has not improved for 2 epochs; restore best weights
Regularisation	dropout 0.5 before the output layer (+0.2 on LSTM embeddings)
Model selection	validation split only; the test split is touched once per experiment
Reproducibility	seed 42 for Python, NumPy and PyTorch; CNN repeated with 3 seeds (Section 3)
Hardware	CPU only (2 cores, no GPU), one thread per run, two runs in parallel
Reference baseline	TF-IDF over the same tokens (uni+bigrams, sublinear tf) + logistic regression (C=10)

Table 2. Training configuration, identical for every experiment unless stated.

What we changed along the way. Our first configuration used 384-token inputs, dropout 0.5 directly on the CNN embeddings and a 128-unit LSTM. The CNN converged slowly (62% validation accuracy after two epochs) and one LSTM epoch took more than six minutes on our CPU, which would not have fit our compute budget. We cut inputs to 256 tokens (covers the median response), removed embedding dropout from the CNN (as in Kim's original model, dropout only before the classifier), and shrank the LSTM to 64 units per direction. Even then a BiLSTM epoch took about 2.5 minutes (8 minutes with prompt + response inputs), so the BiLSTM was used for RQ1, RQ2 and two RQ4 ablations, while the 5-fold RQ3 study used the CNN and the TF-IDF baseline. Every run used the full training schedule, so all comparisons are like-for-like.

3. Results

3.1 RQ1 - Can we identify which LLM generated a response?

Classifier (response only)	Acc.	Macro-F1	F1 ChatGPT	F1 davinci	F1 Cohere	F1 Dolly	F1 BLOOMz
CNN	0.801	0.803	0.75	0.70	0.76	0.83	0.97
BiLSTM	0.756	0.756	0.72	0.65	0.70	0.75	0.96
TF-IDF + LR (baseline)	0.889	0.888	0.89	0.85	0.85	0.86	0.99

Table 3. RQ1 test results (2,234 test examples, 5 classes, chance = 0.20). CNN over 3 seeds: accuracy 0.811 ± 0.009, macro-F1 0.810 ± 0.008.

Yes - clearly. From the response alone the CNN identifies the author 80.1% of the time and the BiLSTM 75.6%, against 20% chance. **BLOOMz is almost perfectly identifiable** (F1 0.97 for the CNN): its outputs are shorter, terser and contain unusual punctuation and repetition (Section 4.2). The hardest pair is the two OpenAI models. 148 of the CNN's 227 mistakes on ChatGPT/davinci examples are confusions between ChatGPT and davinci. If we only ask for the *family*, merging the two OpenAI models, CNN accuracy rises from 80.1% to 86.7% (BiLSTM: 75.6% to 82.4%). Same-family models share a fingerprint.

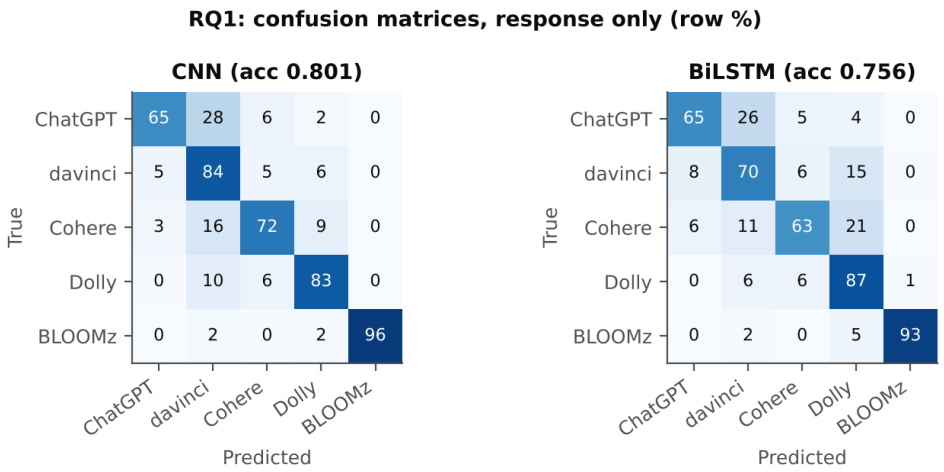


Figure 1. Row-normalised confusion matrices (percent of each true model). For both models the largest error is ChatGPT predicted as davinci; the BiLSTM also often mistakes Cohere for Dolly.

The CNN beat the BiLSTM, and the linear baseline beat both. TF-IDF + logistic regression reaches 88.9%. With about 10.5k training texts and embeddings learned from scratch, a linear model over ~100k sparse n-gram features is very hard to beat, because the fingerprint consists of exactly such n-grams (Section 4.2). The CNN's 3/4/5-gram filters approximate the same thing and train fast. The BiLSTM reads tokens one at a time through a small 64-unit state per direction. It learned more slowly (Figure 2) and ended at a lower validation score, and nothing in this task seems to need its long-range memory: the cues are local. Both neural models eventually overfit (training loss keeps falling while validation F1 plateaus: best epoch 5 for the CNN, 7 for the BiLSTM), which is why early stopping on validation macro-F1 and restoring the best weights matter.

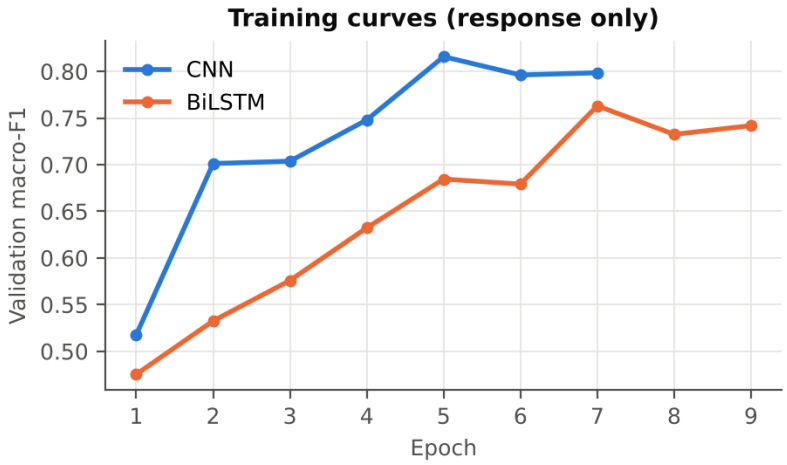


Figure 2. Validation macro-F1 per epoch (response only). Training stops after two epochs without improvement and the best epoch's weights are restored.

3.2 RQ2 - Does the user's prompt help identify the LLM?

Classifier	Input only (prompt)	Output only (response)	Input + Output
CNN	0.720 / 0.725	0.801 / 0.803	0.905 / 0.905
BiLSTM	0.704 / 0.705	0.756 / 0.756	0.857 / 0.854
TF-IDF + LR	0.718 / 0.716	0.889 / 0.888	0.944 / 0.944

Table 4. RQ2 test accuracy / macro-F1 for the three input types.

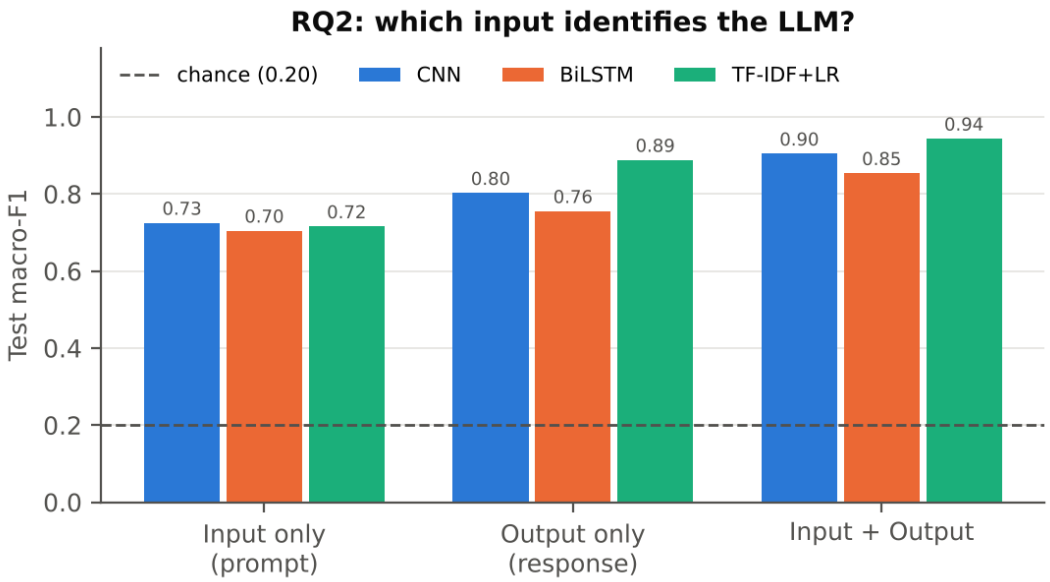


Figure 3. RQ2: macro-F1 by input type. Dashed line = chance.

(i) Can the LLM be predicted from the prompt alone? Surprisingly, yes: the CNN gets 72.0% from the prompt without ever seeing the response. **(ii) Does adding the prompt help?** Yes: input + output beats output alone for every classifier (CNN 80.1% → 90.5%). **(iii) Why does prompt-only work so well?** Because of how the data were collected, not because of anything about the models. We gave every model the same topics, so the topic carries no information. What differs is the *wording of the instruction*: the M4 authors prompted each model with its own template (e.g. for Reddit, ChatGPT and davinci got "I will ask you a question... more than 200 words... in a Reddit answering style", Cohere got "Write a couple of paragraphs of at least 500 characters...", Dolly and BLOOMz got "Write a detailed response... Use approximately N words/tokens").

We verified this. If you group the test set by exact prompt string and always predict the most common model per group, you get the **best accuracy any prompt-only classifier can reach**: 75.1% overall. The CNN's 72.0% sits right under that ceiling, domain by domain (Figure 4). The cleanest case is **Wikipedia**: four of the five models received the *identical* prompt, so a prompt-only classifier can only single out BLOOMz and must guess among the other four. The best possible accuracy is therefore 1/5 + 4/5 × 1/4 = 40%, and the CNN scores exactly 40.0%. On arXiv every model had a different template, so prompt-only accuracy is 100.0%.

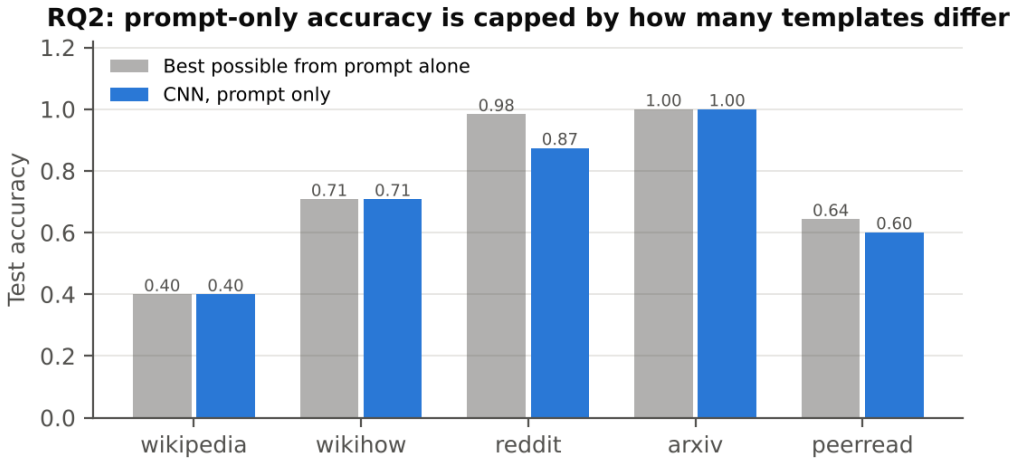


Figure 4. Prompt-only accuracy per domain (CNN) against the best accuracy achievable from the exact prompt string. The classifier has learned the templates, nothing more.

CNN input	wikipedia	wikihow	reddit	arxiv	peerread
input	0.40	0.71	0.87	1.00	0.60
output	0.65	0.84	0.69	0.87	0.92
both	0.66	0.92	0.94	1.00	0.97

Table 5. CNN test accuracy per domain and input type.

Table 5 shows the same effect from the other side. Adding the prompt helps a lot where the templates differ (Reddit +25.1 points, arXiv +13.0) and barely at all on Wikipedia (+1.0), where four models share one template. **Takeaway:** in this benchmark the prompt is a leak of the data-collection protocol. If every model received the same prompt, as in a real attribution setting, prompt-only accuracy would be at chance and any gain from adding the input would disappear. The honest answer to RQ1 is the response-only number.

4. In-Depth Analyses

4.1 RQ3 - Do LLM fingerprints generalise across domains? (extra credit)

Setup. Leave-one-domain-out: for each of the five domains we train on the other four (their train split, early stopping on their validation split) and test on *all 3,000 examples* of the held-out domain, which the model has never seen: different task, different prompt templates, different vocabulary. We compare against the in-domain test accuracy of the RQ1 model on the same domain. We ran this for the CNN and the TF-IDF baseline (10 extra training runs); five more BiLSTM folds would have taken about two more CPU-hours.

Held-out domain	CNN: seen → unseen	TF-IDF + LR: seen → unseen
wikipedia	0.65 → 0.35	0.80 → 0.35
wikihow	0.84 → 0.61	0.91 → 0.68
reddit	0.69 → 0.64	0.82 → 0.68
arxiv	0.87 → 0.51	0.92 → 0.49
peerread	0.92 → 0.69	0.97 → 0.80
Mean drop (points)	23.4	28.4

Table 6. RQ3: accuracy on each domain when it was seen in training (in) vs. held out entirely (out).

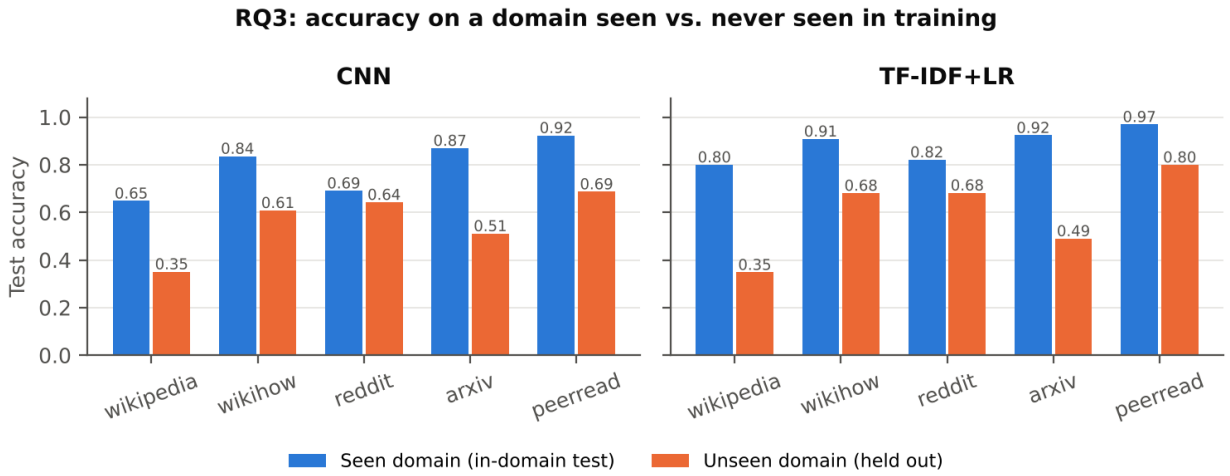


Figure 5. RQ3: in-domain vs. unseen-domain accuracy per held-out domain.

CNN recall on held-out domain	ChatGPT	davinci	Cohere	Dolly-v2	BLOOMz
wikipedia	0.00	0.01	0.03	0.96	0.75
wikihow	0.22	0.89	0.25	0.73	0.95
reddit	0.57	0.52	0.56	0.60	0.96
arxiv	0.42	0.01	0.35	0.85	0.92
peerread	0.57	0.95	0.10	0.83	0.98
mean	0.36	0.48	0.26	0.80	0.91
std (consistency)	0.25	0.46	0.21	0.14	0.09

Table 7. Per-model recall of the CNN on each unseen domain. Low std = consistently identifiable.

(i) Yes, performance drops on unseen domains, sharply and unevenly. Averaged over the five held-out domains, accuracy falls by 23.4 points for the CNN and 28.4 for TF-IDF. On 4 of the 5 unseen domains the CNN stays well above the 20% chance level (51-69%), so part of the fingerprint is domain-general, but on unseen wikipedia it collapses to 35.1% (macro-F1 0.26), barely above chance (explained in (iii)).

(ii) Some models are much more consistently identifiable. Averaged over unseen domains the CNN's recall is BLOOMz 0.91, Dolly-v2 0.80, GPT-3 davinci 0.48, ChatGPT 0.36, Cohere 0.26 (Table 7). BLOOMz and Dolly-v2 keep their identity across tasks. BLOOMz's habits (ellipses, semicolons, terse answers) and Dolly's flat, single-paragraph text appear in every domain. The three API models (ChatGPT, davinci, Cohere) adapt their style to the task much more, so a profile learned on four tasks transfers poorly to a fifth.

(iii) A general fingerprint or task-specific patterns? Both, and the Wikipedia fold shows where the line is. With Wikipedia held out, the CNN labels almost every ChatGPT, davinci and Cohere article as Dolly (recall 0.00 / 0.01 / 0.03, versus 0.96 for Dolly). Wikipedia articles from every model share two traits that, in the four training domains, belong to Dolly. **No line breaks:** in M4's Wikipedia data the ChatGPT, davinci and Cohere articles are a single flat paragraph (0% contain a line break, versus 7-100% in their other domains), exactly like Dolly's text everywhere. **Dates:** Wikipedia articles contain 1.0-2.4 four-digit years per 100 words versus at most 0.28 in the other domains, and Dolly is the model that mentions years most in 3 of the 4 training domains (“2023” is among its top features). A classifier trained on the other domains learns something like “flat text with dates $\rightarrow$ Dolly”, which works there and fails on Wikipedia. We tested the layout half with a follow-up: we re-ran the folds with all layout stripped from every response (the RQ4 *no formatting* transform). For TF-IDF, unseen-Wikipedia accuracy rises from 34.8% to 43.6%, but the CNN does not recover (35.1% to 33.3%) and still sends 1,538 of the 1,800 ChatGPT/davinci/Cohere articles to Dolly. Layout is therefore only part of the shortcut. Meanwhile, on unseen WikiHow removing layout *hurts* (TF-IDF 68.1% to 52.8%; CNN 60.8% to 46.5%), because there every model except Dolly writes step-by-step text with line breaks, so layout is a genuine, transferable cue. So the classifier learns a mix: a **general fingerprint** (punctuation, function words, typography such as davinci's curly apostrophe) that survives a change of task, plus **task- and dataset-specific patterns** (layout, dates, length, task phrases like “this paper”) that do not. Strong in-domain accuracy overstates how well attribution works on a new kind of text.

4.2 RQ4 - What characteristics distinguish the LLMs? (extra credit)

We attacked this from three directions: **(a)** descriptive statistics of 20 hand-crafted stylometric features, **(b)** what the models actually weight, via the strongest n-grams of the linear baseline, and **(c) ablations**: we remove one kind of signal from every response (train and test), retrain, and measure how much performance drops.

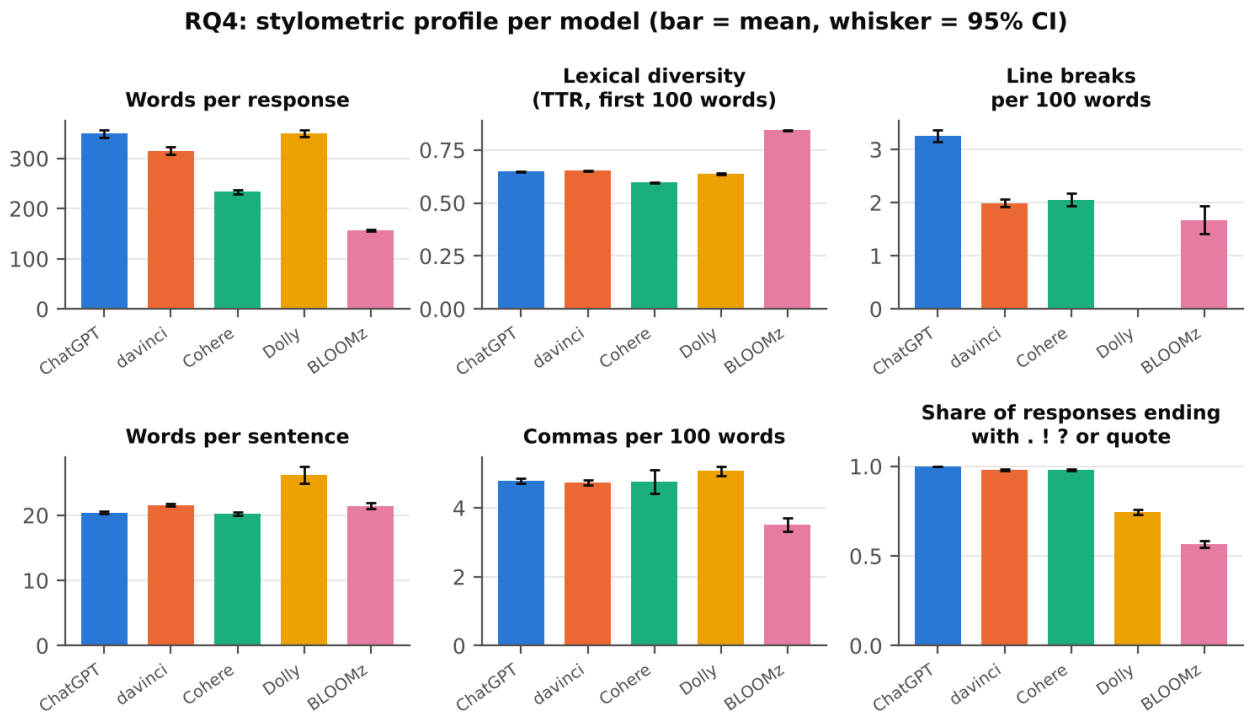


Figure 6. Stylometric profile per model over all 15,000 responses (bar = mean, whisker = 95% confidence interval).

(a) Style statistics. The models differ a lot on simple, measurable properties (Figure 6). BLOOMz writes the shortest answers (mean 157 words vs. 349 for ChatGPT), so its lexical diversity is high. Its mean digit rate (55 per 100 words vs. at most 7 for the others) is driven by a minority of degenerate outputs, e.g. runs of bare list numbers; its median is ordinary. ChatGPT uses the most line breaks and list formatting (on average 10.5% of its lines are list items vs. 2.2% for davinci). Dolly writes the longest sentences (26.2 words) and never breaks lines. A classifier that sees *only these 20 numbers* and no words reaches **82.5%** accuracy with a random forest (68.8% with logistic regression). By permutation importance the most useful features are lexical diversity (whole text) (0.117), line breaks (0.101), length in characters (0.082), length in words (0.082), blank lines (0.073), average word length (0.055). Two caveats: lexical diversity is partly a proxy for length (short texts repeat fewer words), and the line-break features are partly the Dolly collection artifact, which is why we test them with ablations below.

Model	Most indicative n-grams (TF-IDF + LR weights; [NL] = line break)
ChatGPT	“[NL] [NL]”, “[NL]”, “. [NL]”, “significant”, “, which”, “However ,”, “Our”, “However”, “, including”, “Well ,”, “[NL] 2”, “One”
GPT-3 davinci	“”, “[NL]”, “[NL] [NL]”, “an”, “and”, “This paper”, “This”, “’ s”, “-”, “. Additionally”, “, [NL]”, “Additionally ,”
Cohere	“. The”, “, and”, “We”, “a”, “We show”, “The”, “. We”, “the”, “show that”, “was a”, “How to”, “and the”
Dolly-v2	“this”, “for”, “:”, “. I”, “this paper”, “your”, “the first”, “model”, “the time”, “2023”, “paper ,”, “In this”
BLOOMz	“. .”, “;”, “)”, “(”, “during”, “No”, “as”, “/”, “at”, “since”, “September”, “) .”

Table 8. Top-weighted features of the linear baseline (response only).

(b) What the linear model weights. The top features (Table 8) read like a style guide, and each is easy to verify in the data. **ChatGPT** structures its text: paragraph breaks (a blank line appears in 77.8% of its responses), numbered lists, discourse connectives (“However ,”, “, including”, “, which”) and formal hedging vocabulary (“significant”, “various”). **davinci** has a *typographic* tic: it writes the curly apostrophe ’ instead of the ASCII ' in 25.8% of its responses (Cohere: 0.1%), and favours “Additionally ,” and “This paper examines”. **Cohere** writes in an academic first-person plural (“We show that” appears in 11.1% of Cohere responses, under 0.8% for the OpenAI models). **Dolly** is marked by colons, “this paper”, first and second person (“. I”, “your”), explicit years and malformed punctuation (“. ,”). **BLOOMz** uses ellipses (“...” in 26.3% of its responses vs. under 1.5% for every other model), semicolons (34.2%), parentheses, dates and bare “Yes” / “No” answers. The fingerprint is therefore **both lexical** (function words, connectives, pronouns) **and structural** (layout, punctuation, typography).

Ablation (applied to every response)	What it removes	CNN	BiLSTM	TF-IDF
Original	-	0.803	0.756	0.888
No formatting	line breaks, markdown, list markers	0.767	0.686	0.863
Lower-case	capitalisation	0.785	-	0.884
No punct. + no format	all punctuation and layout	0.752	-	0.861
Style only	every content word (kept: 150 most common words, punctuation, layout)	0.810	0.819	0.862
First 50 words	length and everything after the opening	0.674	-	0.731

Table 9. RQ4 ablations: test macro-F1 after retraining on modified responses (BiLSTM run on two ablations only, CPU budget).

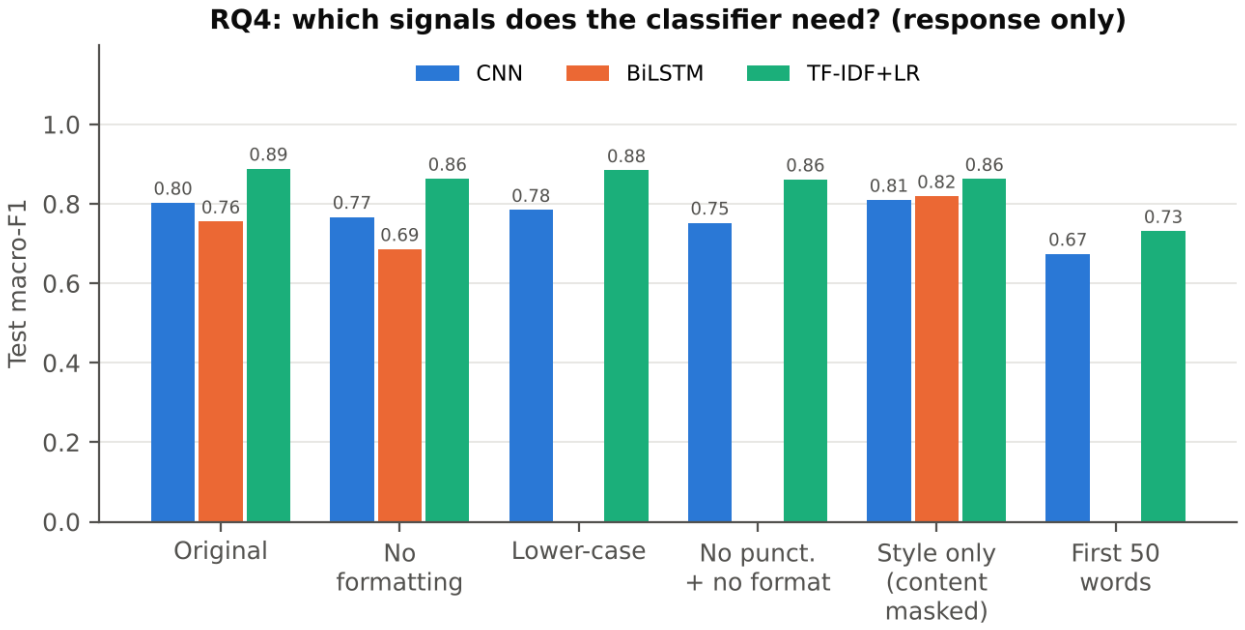


Figure 7. RQ4 ablations (test macro-F1).

(c) Ablations. Removing all formatting (line breaks, markdown, list markers) costs 3.6 macro-F1 points for the CNN and 2.5 for TF-IDF. Layout matters (including the Dolly artifact), but most of the signal survives without it. Removing punctuation as well costs 5.0 / 2.7 points in total, and the models still reach 0.75 / 0.86 from bare words. Lower-casing barely matters (1.8 / 0.4 points). The most revealing test is **style only**: we replace every content word with a placeholder “W” and keep only the 150 most frequent words, punctuation and layout. This costs TF-IDF just 2.6 points (0.86), and the CNN does not lose anything (0.81 vs. 0.80; likely because it can no longer overfit to topic words). **The models identify the author from *how* the text is written, not *what* it says.** The largest drop comes from **truncating every response to its first 50 words**: 0.67 / 0.73 (12.9 / 15.7 points). The opening carries part of the fingerprint, but length and what comes later (lists, paragraphing, closing formulas) carry a lot as well. The BiLSTM (run on two ablations; original 0.76) scores no formatting 0.69 and style only 0.82. Masking content words actually *helps* it (+6.4 points). Plausibly, with topic words gone the small LSTM cannot fit them and must learn style cues that generalise to unseen topics.

Answers. (i) The classifiers use function words and connectives, pronouns, punctuation and typography, layout and length. Topic words contribute little. (ii) The signals are mostly *stylistic*: partly linguistic (function words, connectives), partly structural (punctuation, layout, length). (iii) Only shortening the text removes a large share. Removing any one stylistic signal costs the CNN and TF-IDF at most about 5 points, because the others carry overlapping information. The BiLSTM leans more on layout (7.0 points without formatting). The fingerprint is redundant, spread over many weak cues rather than one tell-tale marker. That makes it robust to simple edits, but also means a few artifacts (Dolly's flattened text, BLOOMz's prompt echo) can inflate scores if they are not

caught.

4.3 Error analysis

Three patterns stand out in the CNN's errors. **First, davinci is the 'default' wrong answer.** It is the most common wrong prediction not only for ChatGPT but also for Cohere and Dolly (Figure 1). Its plain, encyclopedic prose is the most generic style in the set, so texts without strong markers drift towards it. **Second, open-ended prose is hardest.** Accuracy is highest on PeerRead and arXiv and lowest on Wikipedia and Reddit (Table 5): on free-form articles and explanations the models converge on a similar register. Reviews and abstracts follow more constrained formats, which plausibly makes each model's habits (e.g. how it structures a review) stand out. **Third, longer is not easier.** Accuracy shows no clear trend with response length (Figure 8; 0.79 for responses under 100 words, 0.77 for 400+ words). Beyond roughly 200 words the models never see the extra text (inputs are truncated to 256 tokens), and length buckets are confounded with model (most short responses are BLOOMz), so this curve should not be over-read.

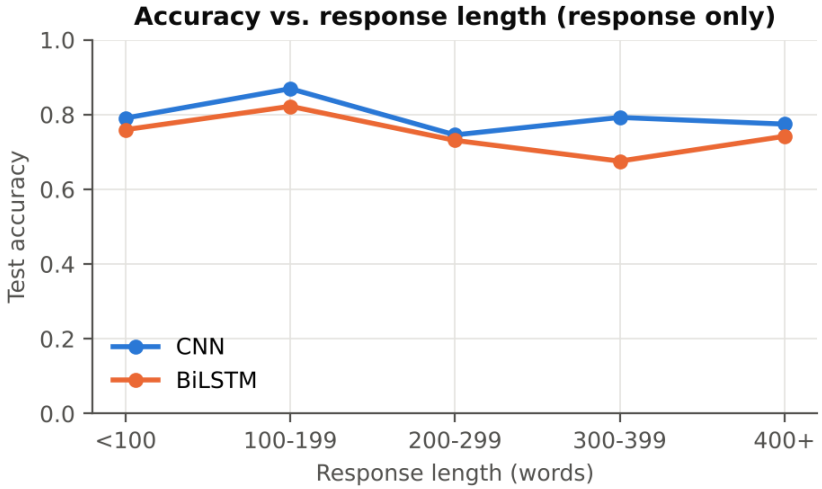


Figure 8. Test accuracy by response length (response-only models).

5. Lessons and Experience

- **Read the raw data before training anything.** Three of our most important findings came from inspecting records, not from models: BLOOMz echoes its prompt (which would have handed us a fake, near-perfect BLOOMz score), 1,780 Cohere Reddit responses are empty, and Dolly's text has had every line break removed. Each would have quietly distorted the results. A fourth artifact (the flattened Wikipedia text of three other models) only surfaced when a model failed in RQ3, so unexpected failures deserve the same scrutiny as raw data.
- **Leakage hides in how data were collected.** RQ2's "surprisingly good" prompt-only classifier is not detecting LLMs at all; it detects which prompt template the dataset authors used for each model. Computing the best accuracy achievable from the prompt string, and breaking results down per domain, turned a confusing number into a clear explanation. We now treat any input that differs systematically between classes as a potential shortcut.
- **Control confounds by design, not after the fact.** Topic-controlled sampling (same topics for all five models) and a split grouped by topic meant topic words could not leak the label and test topics were truly unseen. The Wikipedia case (prompt-only accuracy exactly at the 40% ceiling) shows the control worked.
- **A strong simple baseline is essential.** TF-IDF + logistic regression (88.9%) beat our CNN (80.1%) and BiLSTM on this data size. Without it we would have over-claimed what the neural models achieve. The ablations also explain why: the fingerprint is made of local n-grams and punctuation, which is exactly what a bag-of-n-grams captures.
- **The compute budget shapes the experiment.** We had no GPU. When our first launch was too slow, we profiled one training batch of each model on the CPU. That showed the LSTM would not fit the budget at full

size, so we shortened inputs, shrank the model and planned which runs each model would get. We also made the experiment runner skip finished runs, so parallel job queues could share work and resume safely. Profiling first would have saved us a false start.

- **Accuracy tells you *whether*; ablations tell you *why*.** Removing one signal at a time and retraining was the most informative experiment we ran, because it separated what the models *can* use from what they *need*. It also gave our most surprising result: *removing* information (masking every content word) improved the BiLSTM from 0.76 to 0.82 macro-F1. Less input can mean less to overfit.

Limitations and what we would do next

- The LLMs in M4 are 2023-era models. Current GPT, Claude, Gemini and Llama models are not included, and fingerprints change between model versions.
- Each model was sampled with its own prompts and decoding settings, so part of what we call a "fingerprint" is the generation setup, not the model. A cleaner study would send identical prompts with identical settings to every model.
- Responses were truncated to 256 tokens. On our CPU budget the BiLSTM was trained with one seed and left out of the 5-fold RQ3 study. The CNN was repeated with 3 seeds.
- Next steps: fine-tune a pretrained transformer encoder (e.g. RoBERTa) as a stronger model; add a character-level CNN to capture typographic cues such as the curly apostrophe; test robustness when responses are paraphrased or reformatted.

Appendix A. Reproducing the results

All code is in the GitHub repository. On a CPU-only machine the full pipeline takes about 3 hours (two job queues in parallel).

```
git clone https://github.com/mbzuai-nlp/M4.git
pip install -r requirements.txt
python src/build_dataset.py --m4_dir M4/data --per_cell 600
python src/experiments.py --rq rq1_rq2 # RQ1 + RQ2
python src/experiments.py --rq rq3 --models cnn # RQ3 leave-one-domain-out
python src/experiments.py --rq rq3_noformat --models cnn --domains wikipedia,wikihow
python src/experiments.py --rq rq4 --models cnn # RQ4 ablations
python src/experiments.py --rq rq4_one --models lstm --abl no_format # (and style_only)
python src/experiments.py --rq rq1_seeds --models cnn # extra CNN seeds
python src/style_stats.py # RQ4 style features + prompt templates
python src/make_figures.py && python report/build_report.py
```

Repository layout. *src/build_dataset.py* (data curation), *src/text.py* (tokenizer, input construction, ablations), *src/models.py* (CNN, BiLSTM), *src/train.py* (training loop, early stopping, metrics, TF-IDF baseline), *src/experiments.py* (RQ1-RQ4 runs), *src/style_stats.py* (stylometry), *src/make_figures.py*, *results/* (one JSON per run with metrics, confusion matrix and training history), *figures/*, *report/*.

Reference. Y. Wang et al. "M4: Multi-generator, Multi-domain, and Multi-lingual Black-Box Machine-Generated Text Detection." EACL 2024. Y. Kim. "Convolutional Neural Networks for Sentence Classification." EMNLP 2014.

Use of AI tools. As permitted by the project guidelines, we used an AI assistant (Claude) to help write and debug code and to draft parts of this report. We reviewed the code, re-ran every experiment, and checked that every number in this report is generated from the saved result files.